

Modul Praktikum

Algoritma dan Struktur Data (TPL2106)

Oleh:

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

Teknologi Rekayasa Perangkat Lunak
Semester Genap
Tahun Akademik 2025/2026

Modul 6

Tree II : Binary Search Tree (BST) dan AVL Tree

1. Deskripsi Singkat Modul

Modul 6 merupakan kelanjutan dari pembelajaran struktur data Tree yang telah dibahas pada Modul 5. Pada modul ini, mahasiswa akan mempelajari Binary Search Tree (BST) sebagai bentuk Tree yang memiliki **aturan khusus** dalam penyimpanan data, serta AVL Tree sebagai pengembangan BST yang mampu menjaga keseimbangan Tree secara otomatis.

Mahasiswa akan memahami bagaimana data diorganisasikan dalam BST berdasarkan aturan perbandingan nilai, serta bagaimana operasi dasar seperti insert dan search dilakukan. Selanjutnya, mahasiswa akan mempelajari konsep balanced tree melalui AVL Tree, termasuk mekanisme rotasi untuk menjaga keseimbangan Tree.

2. Capaian Pembelajaran

Setelah mengikuti praktikum pada Modul 5 ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep Binary Search Tree (BST).
2. Mengimplementasikan operasi dasar BST (insert dan search).
3. Menganalisis struktur BST berdasarkan urutan data.
4. Menjelaskan konsep keseimbangan pada Tree.
5. Memahami prinsip kerja AVL Tree.
6. Mengidentifikasi kondisi unbalanced pada Tree.
7. Menjelaskan dan mengimplementasikan rotasi sederhana pada AVL Tree.
8. Menulis kode program BST dan AVL Tree secara terstruktur dan terdokumentasi.

3. Prasyarat Pengetahuan Awal

Sebelum mengikuti praktikum pada modul ini, mahasiswa diharapkan telah memahami:

- Konsep dasar Tree
- Binary Tree dan traversal
- Rekursi dalam Python
- Struktur Node (left dan right)
- Perbandingan nilai (comparison).

4. Kebutuhan Alat

A. Perangkat Keras

Laptop/PC dengan spesifikasi minimum:

- RAM minimal 4 GB (disarankan 8 GB)
- Ruang kosong minimal 1 GB

Koneksi internet (opsional) untuk instalasi awal dan update extension.

B. Perangkat Lunak

- Python 3.x (disarankan Python 3.10+)
- Visual Studio Code (VS Code)
- Extension VS Code: Python (Microsoft)
- Windows Command Prompt / PowerShell / Git Bash

C. Folder kerja praktikum

5. Instruksi Ketentuan Teknis Praktikum

- a) Mahasiswa wajib menyimpan seluruh file praktikum dalam repository Github masing-masing sesuai dengan ketentuan yang berlaku.
- b) Penamaan file mengikuti instruksi modul
Praktikum10-latihan1.py s/d praktikum10-latihan4.py
- c) Setiap kode program harus diberi **komentar** untuk menjelaskan tujuan

blok program sebagai pembiasaan dokumentasi kode. Serta gunakan komentar pada program terkait untuk menjawab pertanyaan-pertanyaan pada latihan

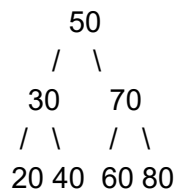
6. Materi

6.1 Konsep Binary Search Tree (BST)

Binary Search Tree (BST) adalah jenis Binary Tree yang memiliki aturan:

Semua nilai di kiri < root < semua nilai di kanan

Perhatikan ilustrasi BST berikut ini :



Root = 50
Root.left = 30
Root.left.left = 20
Root.left.right = 40
Root.right = 70
Root.right.left = 60
Root.right.right = 80

- 30 < 50 maka masuk kiri
- 70 > 50 maka masuk kanan
- 20 < 30 maka kiri dari 30

Semua mengikuti aturan perbandingan.

Struktur node BST sebagai berikut :

```
# =====
# Node untuk BST
# =====

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

6.2 Operasi Insert Pada BST

Konsep

- Jika data < node maka ke kiri
- Jika data > node maka ke kanan
- Ulangi sampai posisi kosong

Contoh implementasi operasi insert pada BST

```
# =====
# Insert pada BST
# =====

def insert(root, data):
    # Jika root kosong, buat node baru
    if root is None:
        return Node(data)

    # Jika data lebih kecil → ke kiri
    if data < root.data:
        root.left = insert(root.left, data)

    # Jika data lebih besar → ke kanan
    elif data > root.data:
        root.right = insert(root.right, data)

    return root
```

Penggunaannya :

```
root = None

data_list = [50, 30, 70, 20, 40, 60, 80]

for data in data_list:
    root = insert(root, data)
```

6.3 Operasi Search Pada BST

Konsep

- Jika data = root maka ditemukan
- Jika data < root maka cari kiri
- Jika data > root maka cari kanan

Contoh Implementasi Search pada BST

```
# =====
# Search pada BST
# =====

def search(root, key):
    if root is None:
        return False
```



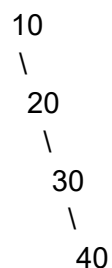
```
if root.data == key:
    return True

elif key < root.data:
    return search(root.left, key)

else:
    return search(root.right, key)
```

6.4 Kelemahan BST

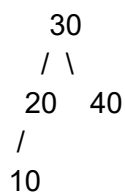
BST bisa menjadi tidak seimbang jika bentuk datanya seperti ini



Ini seperti Linked List maka tidak efisien

6.5 Konsep AVL Tree

AVL Tree adalah BST yang selalu seimbang. Aturan: Selisih tinggi subtree kiri dan kanan maksimal = 1. Ilustrasi AVL Tree :



6.6 Konsep Balance AVL Tree

Konsep Balance Factor : Balance Factor = tinggi kiri - tinggi kanan

Kondisi	Status
-1, 0, 1	Seimbang
>1 atau <-1	Tidak seimbang

6.7 Rotasi pada AVL Tree

Untuk menjaga keseimbangan, digunakan rotasi:



1. Rotasi Kanan (Right Rotation - LL)

```
  30
 /
20
/
10
```

Menjadi

```
  20
 / \
10  30
```

Implementasi Rotasi Kanan

```
def rotate_right(y):
    x = y.left
    T2 = x.right

    # Rotasi
    x.right = y
    y.left = T2

    return x
```

2. Rotasi Kiri (Left Rotation - RR)

```
  10
 \
  20
 \
  30
```

Menjadi:

```
  20
 / \
10  30
```

Implementasi Rotasi Kiri

```
def rotate_left(x):
    y = x.right
    T2 = y.left

    # Rotasi
    y.left = x
    x.right = T2

    return y
```

6.8 Insert pada AVL Tree (Konsep Sederhana)



Langkah:

1. Insert seperti BST
2. Hitung balance factor
3. Jika tidak seimbang → lakukan rota

Struktur Dasar AVL Node

```
class AVLNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1
```

6.9 Perbandingan BST vs AVL

Aspek	BST	AVL
Struktur	Bisa tidak seimbang	Selalu seimbang
Search	Bisa lambat	Lebih cepat
Kompleksitas	$O(n)$	$O(\log n)$

7. Latihan – Latihan

Latihan 1 . Membuat Node dan BST

```
# =====
# Latihan 1: Node dan Insert BST
# =====

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

# Membuat BST
root = None
```




```
data_list = [50, 30, 70, 20, 40, 60, 80]

for data in data_list:
    root = insert(root, data)

print("BST berhasil dibuat")
```

Latihan 2 . Traversal Inorder (Sorting Otomatis)

```
# =====
# Latihan 2: Traversal Inorder
# =====

def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

print("Hasil inorder:")
inorder(root)
```

Latihan 3 . Searching

```
# =====
# Latihan 3: Search BST
# =====

def search(root, key):
    if root is None:
        return False

    if root.data == key:
        return True

    elif key < root.data:
        return search(root.left, key)

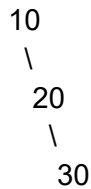
    else:
        return search(root.right, key)

# Uji pencarian
key = 10

if search(root, key):
    print("Data ditemukan")
else:
    print("Data tidak ditemukan")
```

Latihan 4 . Membuat BST yang Tidak Seimbang

Ilustrasi kasus . Jika data dimasukkan berurutan 10, 20, 30, maka BST menjadi



Tree ini **tidak seimbang** karena semua node condong ke kanan. Coba pelajari kode program berikut ini :

```
# =====
# Latihan 4: Membuat BST yang Tidak Seimbang
# =====

# Class Node untuk menyimpan data BST
class Node:
    def __init__(self, data):
        self.data = data      # nilai pada node
        self.left = None     # child kiri
        self.right = None    # child kanan

# Fungsi insert untuk BST
def insert(root, data):
    # Jika root kosong, buat node baru
    if root is None:
        return Node(data)

    # Jika data lebih kecil, masuk ke subtree kiri
    if data < root.data:
        root.left = insert(root.left, data)

    # Jika data lebih besar, masuk ke subtree kanan
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

# Fungsi preorder untuk melihat bentuk tree
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

# Fungsi sederhana untuk menampilkan struktur tree
def tampil_struktur(root, level=0, posisi="Root"):
    if root is not None:
        print("  " * level + f"{posisi}: {root.data}")
        tampil_struktur(root.left, level + 1, "L")
```



```
tampil_struktur(root.right, level + 1, "R")
```

```
# -----
# Program utama
# -----
root = None

# Data dimasukkan berurutan naik
data_list = [10, 20, 30]

for data in data_list:
    root = insert(root, data)

print("Preorder BST:")
preorder(root)

print("\n\nStruktur BST:")
tampil_struktur(root)
```

Output yang diharapkan:

```
Preorder BST:
10 20 30
```

```
Struktur BST:
Root: 10
  R: 20
    R: 30
```

Checkpoint

Mahasiswa mampu menjelaskan bahwa:

- tree condong ke kanan
- semakin panjang tree, pencarian bisa makin lambat
- BST tidak selalu seimbang

Latihan 5 Rotasi Kiri pada BST Tidak Seimbang

Ilustrasi Kasus

Sebelum rotasi kiri :

```
10
 \
  20
  \
   30
```

Sesudah rotasi kiri

```
  20
 / \
10  30
```

Kode program untuk implementasi :

```
# =====
# Latihan 5: Rotasi Kiri pada BST Tidak Seimbang
# =====

# Class Node
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Fungsi preorder untuk melihat isi tree
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

# Fungsi untuk menampilkan struktur tree
def tampil_struktur(root, level=0, posisi="Root"):
    if root is not None:
        print(" " * level + f"{posisi}: {root.data}")
        tampil_struktur(root.left, level + 1, "L")
        tampil_struktur(root.right, level + 1, "R")

# Fungsi rotasi kiri
def rotate_left(x):
    # x adalah root lama
    y = x.right      # y adalah child kanan x
    T2 = y.left      # subtree kiri milik y disimpan sementara

    # Proses rotasi
    y.left = x       # x menjadi child kiri dari y
    x.right = T2      # child kanan x diganti dengan T2

    # y menjadi root baru
    return y
```



```
# -----  
# Program utama  
# -----  
  
# Membuat tree yang tidak seimbang:  
# 10 -> 20 -> 30  
root = Node(10)  
root.right = Node(20)  
root.right.right = Node(30)  
  
print("Preorder sebelum rotasi kiri:")  
preorder(root)  
  
print("\n\nStruktur sebelum rotasi kiri:")  
tampil_struktur(root)  
  
# Melakukan rotasi kiri pada root  
root = rotate_left(root)  
  
print("\nPreorder sesudah rotasi kiri:")  
preorder(root)  
  
print("\n\nStruktur sesudah rotasi kiri:")  
tampil_struktur(root)
```

Latihan 6 . Rotasi Kanan pada BST Tidak Seimbang

Data : 30,20,10

Sebelum Rotasi Kanan

```
  30  
 /  
20  
/  
10
```

Sesudah Rotasi Kanan

```
  20  
 /  \  
10  30
```

TUGAS : Buatlah kode programnya untuk kasus ini (di Latihan 6).

Ketentuan Praktikum

Buatlah semua program latihan di atas dan masing-masing latihan disimpan dalam 3 file yang berbeda dengan nama file praktikum10-latihan1.py (Gabungan latihan 1-3) sampai dengan praktikum10-latihan4.py. Setiap file diberikan header identitas mahasiswa pada bagian atas kode program. Diskusikan kemudian tulis penjelasan dan



IPB University
— Bogor Indonesia —

Sekolah
Vokasi

hasil diskusi pada file program sebagai komentar. Semakin jelas dan lengkap akan semakin bagus. Kumpulkan di akhir praktikum di repo masing-masing pada folder praktikum10.